

Project Reference Notes (Gevis SRL + AI Cycle)

This document captures your project history **exactly as you described it** (no assumptions), organized into clear sections for future CV/interview writing.

Company Context

- **Company:** Gevis SRL (industrial vision systems company)
 - **Baseline before you joined:** Traditional vision software/tools, **no AI culture / no AI in product.**
 - **Why you were hired:** CEO wanted AI to improve accuracy and future capability.
 - **Your overall contribution:** Started AI culture from **zero to production**, built datasets/training/evaluation approach, delivered models (typically exported to **ONNX**), collaborated with the team that integrates into the **C++ vision application**, and later initiated/owned an internal platform (AI Cycle) to operationalize the full iteration loop.
-

Project 1 — Bread/Toast Defect Detection (Production)

Goal

Detect defects on bread/toast images across ~**6 categories**. Defects include things like holes/break/burn/size defects (as described).

Reality / Constraints

- Defects are **very small**.
- Image quality was **not good**.
- Camera/resolution were already installed in production → **could not change resolution/camera**.
- Needed to succeed with the existing acquisition setup.

What you did (Ownership)

- First AI project after joining; you built the workflow from scratch.
- Defined how to **capture** images and how to **label**.
- Built dataset from limited/low-quality images.
- Image engineering:
 - Isolated defects (as possible)
 - Reshaped/cropped to reduce wasted area and improve training signal
 - Used augmentation to increase usable variety
 - Trained the model and iterated on failure modes.

Key Problems Solved

- Confusion between defect types (example: **hole vs break** leading to false positives).
- Improved performance through iterations over time.

Deployment / Integration

- You **delivered the model** (export to **ONNX**).
- **Integration into the vision tool was done by others** (C++ tool modified to connect/load your ONNX model).
- You supported integration by communicating constraints and requirements.

Outcome

- System has been running in production for **~2 years** without operational remarks.
-

Project 2 — Industrial Part Anomaly Detection (Production)

Goal

Detect subtle anomalies and missing material on an industrial part (rubber + metal combination / bumper-like component). Task is more like “**clean vs defect**” **anomaly detection** rather than multi-class categorization.

Baseline System

- Traditional vision analysis was inspecting a **ring region**, but could not reliably decide defect vs clean.

Data Reality

- Very imbalanced:
- **~2000** clean images
- only **~100–150** real anomaly images
- Noise is high; images look similar except for **tiny defect differences**.

Your Key Technical Decision

- You concluded object detection/segmentation was not ideal given low anomaly count.
- You researched and introduced an **anomaly detection approach/library** suitable for limited defect data.

What you did (Engineering)

- Major issue discovered: anomaly detection is **very sensitive** to irrelevant pixels/background.
- You used the existing vision tool to capture only the **important analysis area (ROI)**.
- You transformed/normalized the ROI:
- ring ROI was reshaped to a consistent representation to reduce variation.
- Trained the anomaly model using pixel/feature comparison style (you referenced a pixel-to-pixel comparison approach and “ActiveNets” context).

Deployment / Adoption

- Deployed and used in production inside your company’s existing app.
- You worked with your team members and the customer to present results and bring it into real use.

Project 3 — Metal Pad (“Red Pad”) Defect Detection (Production)

Goal

Detect multiple defect types on a metal pad using grayscale images with heavy noise.

Reality / Constraints

- Images are mostly **grayscale**.
- Two ring ROIs:
 - **big ring + small ring**
- Input images were large (~**2500×2500**), with lots of empty/black area.
- Defects included (as described): scratches, small/large contamination, spot/holes-like defects, etc.
- Low defect data initially (~**200** defect images across ~4-5 categories).

Your Approach (Phased + Honest)

1) **Phase 1 (low data)**: - You convinced CEO + customer to start with **single-class** training first (all defects treated as one class, “Reject”), because multi-class would bias/overfit. 2) **Ring normalization / preprocessing**: - With the team, you developed a method to: - extract big/small rings - **unwrap/open the ring into a horizontal strip** (ring → line) - align big + small strip representations - adjust small ring width using overlap/stretching to match reference width - Final representation: combined strips into a consistent image (~300px height; width ~2500px) then sliced into square inputs. 3) **Model input constraint handling**: - Because CNNs prefer square inputs, you used slicing/overlap to create manageable tiles. 4) **False positive root-cause**: - Serial number / stamped markings caused FP (looked like defects). - You introduced a nuisance handling strategy (separate class or suppression), and also planned to hide those predictions if needed. 5) **Critical training insight**: - Training only on defect images made the model treat “noise” as defect. - You added **clean (good) images with no defect labels** as negative context in train and validation.

Model Evolution

- Initial training/deployment: **YOLOv8 → ONNX**
- Later multi-class training (after collecting more defect samples ~**60-70 per class**) and optimization: migrated to **YOLOv9**.

Outcome

- Multi-class model achieved ~**95%** defect detection performance (as you stated) and reduced null-field false positives.
- Deployed in production.

Project 4 — Gasket Inspection System (Production)

Product Context

- **Three different gasket models** (different shapes/design).

- At the start, nobody knew the real defect appearance well; design was assumed perfect but production reality differed.

Inspection Requirements

- Need to analyze multiple surfaces:
- inside / outside / top / bottom
- Some areas include black rubber regions; visibility and lighting are challenging.

Capture / System Challenges

- Needed custom capture setup:
- small mirrors to capture inside/outside areas
- part rotation
- overlapping captures
- Image sizes were inconsistent and could be very large (examples you gave: ~1500, 2000, up to ~8000 pixels).

What you owned

- You led the end-to-end effort to make images usable for AI and inspection:
- guided camera/lighting changes, FPS adjustments
- decided what constitutes **good AI training images**
- removed wasted space, reshaped into square images
- normalized sizes (multiples of **32**) to fit model constraints

Core Technical Backbone (Confirmed)

- Main inspection logic relied on **classic CV segmentation** inside the vision tool:
- thresholding, morphology, contours, masking
- You experimented with deep segmentation (U-Net / Mask R-CNN), but with low data it **overfit**, so it was rejected.

AI Strategy

- You iterated through detector versions (**YOLO10** → **YOLO11**) for AI-assisted screening.
- Because defect data was scarce/noisy, you started with **single-class** (defect present) rather than full defect categories.

Production Purpose (Important)

- Not positioned as “perfect final accuracy.”
- Used as a **defect harvesting loop**:
- run in production to surface possible defects
- collect more defect images
- later retrain with defect categories and also measurement/tolerance rules.

Outcome

- In production, useful for finding defects and accelerating dataset growth.

Project 5 — Sport Car Brake Component Crack Detection (R&D / Pilot)

Goal

Detect small cracks / pothole-like surface defects on a large circular brake component.

Capture Reality

- Five different filtered/angle images per sector.
- Around the circle:
 - **24 sectors**
 - each sector has **5 views**
 - raw total = **24 × 5 images per part**
 - Each image is very high resolution (~5000×5000).
 - Crack visibility can be weak; noise and pressing/production conditions make it hard.

Key Engineering Steps

1) **Multi-view fusion:** - You and team fused the 5 filtered views into a single representation (converted multi-view grayscale into an RGB/pseudo-color composite), reducing per-part images from 24×5 to **24 fused images**. 2) **Why cropping wasn't enough:** - Cropping/resizing would miss small cracks; full-size training was not feasible. 3) **Tiling strategy + class imbalance control:** - You used a tiling approach: - split each large image into **12 or 16 tiles** - resize tiles to **640×640** - Avoided a dataset dominated by empty/good tiles: - selected tiles from defect images - filtered out all-good tiles - injected random good tiles with controlled ratio (~**2:1** defect:good) - split into train/validation (you mentioned 50/50 example in one scenario) 4) **SAHI slicing inference on full images:** - For inference you used slicing/scanning on the full 5000×5000 image: - window size ~640×640 - overlap - IoU-based merging / duplicate removal - final reconstruction overlays on full image 5) **Confidence/IoU tuning:** - Worked on selecting suitable confidence and IoU settings for practical performance.

Status

- R&D / pilot project (not in production).
-

AI Cycle App — Internal CV Platform (Active Development)

Motivation

- CEO initial expectation: simple loop: **label → train → model**.
- Your view: real industrial pain is evaluation, iteration, data lifecycle, and deployment decision clarity.
- You had **zero web-app knowledge** at start; no one internal could build it for your vision.
- You learned step-by-step and used a coding agent to deliver the product vision.

Tech Stack (Confirmed)

- **Frontend:** React + Ant Design
- **Backend:** Python + FastAPI

- **Training:** Ultralytics YOLO + custom scripts
- **Database:** relational DB with ~**10-15 tables** (as you stated)

Scope (What the platform includes)

1) Data ingestion & management

- Upload images/video
- Structured project/dataset organization
- Stable state handling + debugging

2) Labeling and AI-assisted annotation

- Systematic labeling workflows
- AI-assisted segmentation/annotation support
- Manual edits and corrections saved persistently

3) Dataset engineering & release generation

- Train/Val/Test split management
- Rebalancing controls
- Null image handling
- Duplicate detection
- Image hashing/ID system for consistency

4) Augmentation engine (configurable)

- ~18 automation tools and options (as you stated)
- Dynamic chained augmentation logic (example you gave):
original → +25 contrast → crop → -25 contrast → crop → ...
- User-controlled generation counts per original image

5) Training orchestration

- Full training UI with configuration groups
- Presets and reuse of configurations for future training
- Training executed via subprocess/terminal to avoid blocking main server
- Automated metric capture into DB

6) Prediction & review workflow (core value)

- Inference section that saves predictions and overlays
- Rich filtering:
 - confidence / IoU / size / duplicates
 - image-based filters (detection count, etc.)
- Review system:
 - risk tagging (low/medium/high)
 - approve/reject
 - manual drawing tools
 - corrections stored for next training

7) Evaluation & analytics (your direction)

- FP/FN hinting and error visibility
- Compare different model runs/predictions within the same project
- “Analytical insights” section to explain what’s wrong and how to improve
- Performance Decision Engine work (confidence/IoU, minimum automation floor, model ceiling, retraining blueprint)

Current scale (as you stated)

- Codebase ~**150k lines**
 - ~**120** UI/backend files
 - Still actively working and improving UX, stability, and advanced analytics.
-

Notes for Future CV/Interview Building

- Your consistent strengths across projects:
- Taking ownership from “unknown problem” → working solution
- Designing **capture + ROI normalization** when data quality is the bottleneck
- Being honest about low-data limitations and phasing delivery
- Building reliable iteration loops (defect harvesting / retraining)
- Explaining tradeoffs to CEO/customer and aligning expectations
- Confirmed recurring constraints:
- fixed cameras/resolution in production
- grayscale imagery
- ring-shaped ROIs
- low defect counts and high noise
- need for ONNX delivery into a C++ vision application